Policy & Standard Operating Procedure		Devsinc
Title: Software Development Lifecycle (SDLC) Process	Issued By: HA	

Effective Date:	November 01, 2025
Version:	# 001

Process Owner:	**Final Reviewed by:**	**Approved by:**
Ashar Ali (Associate Project Manager) Signature	CDO Signature	CEO Signature

Revision History			
Rev. No.	Description	Rev. Date	Effective Date
01	SOP for Software Development Lifecycle (SDLC) Process		01-Nov-205

Policy & Standard Operating Procedure		Devsinc
Title: Software Development Lifecycle (SDLC) Process	Issued By: HA	

Introduction

This document outlines a comprehensive software development lifecycle process designed to achieve higher quality and productivity across all development teams. The process is based on Agile methodologies and covers all aspects of software development from initial requirements through post-production support.

Key Objectives:

- Produce high-quality software that meets stakeholder requirements
- Deliver on time and within budget
- Ensure maintainability and scalability
- Generate metrics for continuous improvement
- Maintain security and compliance standards

Development Methodologies

Agile SCRUM

Best For: Complex workflows, larger teams (10+), predictable delivery

Key Features:

- Sprint-based development with fixed time boxes
- High customer involvement through sprint reviews
- Regular retrospectives for continuous improvement
- Defined roles (Product Owner, Scrum Master, Development Team)

Process Flow:

1. Product Backlog creation and prioritization
2. Sprint Planning with story selection
3. Daily Scrum meetings for synchronization
4. Sprint execution with deliverables
5. Sprint Review for stakeholder feedback
6. Sprint Retrospective for process improvement

Agile KANBAN

Best For: Simple workflows, flexible priorities, continuous flow

Key Features:

- Visual workflow management
- Work in Progress (WIP) limits
- Continuous delivery without fixed iterations
- Pull-based system for task management

Process Flow:

1. Visualize work on Kanban board
2. Limit WIP to prevent bottlenecks
3. Manage flow continuously
4. Make process policies explicit
5. Implement feedback loops
6. Improve collaboratively

Selection Criteria

Choose SCRUM when:

- Complex workflows with interdependent tasks
- Team size exceeds 10 members
- Predictability in delivery timeline is needed
- High customer involvement is required
- Work priority needs to be set at sprint beginning

Choose KANBAN when:

- Simple workflows with parallel tasks
- Smaller teams with flexible structure
- Changing requirements are common
- Continuous flow of work is needed
- Less customer involvement is acceptable

SDLC Phases

Phase 1: Requirements & Design

Owner: Delivery Lead, Business Analyst
Participants: Client stakeholders, Development Team

Key Activities

1. **Requirements Gathering & Definition**
    - Identify stakeholder needs and expectations
    - Document functional and non-functional requirements
    - Conduct a formal assessment to identify technical, security, and delivery risks before development begins (ISO Requirement).
    - Categorize all data (e.g., PII, Confidential, Public) to determine required security controls

2. **Story Writing**
    - Follow INVEST principles (Independent, Negotiable, Valuable, Estimatable, Small, Testable)
    - Include clear acceptance criteria
3. **Validation & Approval**
    - Development team validation
    - Stakeholder approval process
    - Review and feedback incorporation
4. **UI/UX Design**
    - Create wireframes and mockups
    - Develop interactive prototypes
    - Stakeholder review and approval
5. **Architecture & Design**
    - Technology selection and evaluation

- Proof of concept development (optional)
- Business flow diagrams
- Solution architecture design
- Context, functional, and component diagrams
- Deployment/physical architecture

6. **Secure Project Management**

    - Establish secure coding processes
    - Define change management procedures
    - Document security requirements

Deliverables

- Product Backlog
- User Stories with acceptance criteria
- Architecture Documents
- UI/UX Prototypes
- Security Requirements
- Technology Stack Documentation

Phase 2: Planning & Setup

Owner: Delivery Lead, SCRUM Master
Participants: Development Team, QA Team, DevOps Team

Key Activities

1. **Resource Planning**

    - Identify required resources (people, tools, infrastructure)
    - Manage internal and external dependencies
    - Create resource procurement timeline

2. **Tool Selection**

- Select project management tool (Jira/Azure DevOps/ClickUp)
- Configure workflows and metadata
- Onboard team members with proper access

3. **Backlog Grooming & Prioritization**

- Review and update Product Backlog
- Break down large stories into manageable ones
- Estimate effort required
- Add new items as needed
- Prioritize based on business value

4. **Sprint Planning**

- Define sprint length and schedule
- Create sprints in project management tool
- Establish sprint goals

5. **Story Estimation**

- Use Story Points approach
- Conduct planning poker sessions
- Apply consistent estimation model (e.g., 1 SP = 0.5 day)

6. **Architecture Review**

- Submit architecture for ARB review
- Incorporate feedback
- Get final approval

Deliverables

- Sprint Plan

- Resource Allocation Matrix
- Groomed and Prioritized Backlog
- Estimation Guidelines
- Approved Architecture

Phase 3: Change Management Policy

Owner: Delivery Lead
Participants: All team members

1. **Standard Change Workflow:**
    - **Request (CR):** All changes must be logged in the project management tool with a unique ID.
    - **Impact Assessment:** Evaluate risks to security, performance, and functionality.
    - **Approval:** Changes require formal approval from the Product Owner or CAB.
    - **Traceability:** All code commits must reference the approved Change Request ID.

2. Emergency Change (Hotfix) Workflow:

- **Criteria:** Critical production bugs, security breaches, or service outages.
- **Process:** Expedited approval (verbal/chat) permitted from Delivery Lead.
- **Post-Implementation Review:** A formal review and documentation update must occur within 72 hours of deployment.

Phase 4: Development (Agile)

Owner: Development Team
Participants: All team members

Key Activities

1. **Sprint 0 (Optional Bootstrap Sprint)**

    - Technology research and evaluation
    - Proof of concept implementation
    - Existing code reviews
    - Local development environment setup

2. **Daily SCRUM Meetings**

    - What was done yesterday
    - What will be done today
    - Any blockers or impediments

3. **Feature Development**

    - Implement user stories according to acceptance criteria
    - Follow coding standards and best practices
    - Write clean, maintainable code

4. **Unit Testing**

    - Write unit tests for all functions
    - Achieve high code coverage (70%+ recommended)
    - Integrate tests into CI/CD pipeline

5. **Security Implementation**

    - **Multi-Factor Authentication (MFA)** implementation
    - **Password Management** with security policies:
        - Minimum length requirements
        - Complexity requirements
        - Password expiration policies
        - Password history tracking
    - **Session Management**:
        - Session timeout configuration (15-30 minutes idle)
        - Automatic session termination
        - Secure session token handling

- **Error Handling & Logging**:
  - Comprehensive error handling
  - Log retention period: 90+ days minimum
  - Secure log storage and access

6. **Pull Requests & Code Reviews**

   - Create PR when feature is complete
   - **Security Code Review** checklist:
     - Authentication and authorization checks
     - Input validation
     - SQL injection prevention
     - XSS prevention
     - Sensitive data handling
     - Cryptography implementation
   - Code quality review
   - Merge after approval

7. **Sprint Closure**

   - Sprint review meeting
   - Sprint retrospective
   - Carry over incomplete stories
   - Update metrics

8. **Secure Coding Practices**

   - Follow OWASP Top 10 guidelines
   - Input validation and sanitization
   - Output encoding

- Parameterized queries
- Secure configuration management

9. **Formal Change Management Process**
    - Every change (bug fix or feature) must be linked to a unique ID in the PM tool.
    - Formal approval is required from the Delivery Lead or Change Advisory Board (CAB) before deployment.
    - Code commits must reference the approved Change Request ID.
    - Handled via an expedited workflow but require a "Post-Implementation Review" within 72 hours.

Deliverables

- Working Software Increment
- Unit Tests with Coverage Reports
- Security Code Review Reports
- Sprint Reports
- Updated Documentation

Phase 5: Testing Phase

Owner: QA Team
Participants: Development Team, Client Stakeholders (for UAT)

Key Activities

1. **Test Planning**
    - Create comprehensive test plan
    - Define scope, objectives, and schedule
    - Identify test environments
    - Document test approach and methodologies
    - Define roles and responsibilities
    - Plan test data requirements

2. **Test Plan Review**

    - Development team review
    - Client stakeholder review (optional)
    - Incorporate feedback
    - Get approval

3. **Test Case Writing**

    - Write test cases for acceptance criteria
    - Include test steps and expected results
    - Map to user stories
    - Cover positive and negative scenarios

4. **Test Case Review**

    - Development team review
    - Stakeholder review for critical cases
    - Update based on feedback

5. **Test Environment Setup**

    - Configure test environment
    - Prepare test data and scripts
    - Verify environment readiness
    - Conduct smoke tests

6. **Manual Functional Testing**

    - Execute test cases manually

- Validate UI/UX functionality
- Test business logic
- Exploratory testing

7. **Automated Testing**

- Write automated test scripts (Selenium, Appium)
- Integrate with CI/CD pipeline
- Execute regression test suites
- Maintain test automation framework

8. **Integration Testing**

- Test component interactions
- Database integration testing
- Third-party API testing
- End-to-end workflow testing

9. **Regression Testing**

- Execute regression test suite
- Verify existing functionality
- Test impacted areas after changes
- Automated regression where possible

10. **Performance Testing**

- Load testing under expected workload
- Stress testing under peak conditions
- Scalability testing
- Response time measurement

- Resource utilization monitoring

11. **Security Testing**

- **Authentication & Authorization Testing**:
  - MFA functionality verification
  - Password policy enforcement
  - Role-based access control
- **Session Management Testing**:
  - Session timeout verification
  - Session termination testing
  - Concurrent session handling
- **Vulnerability Scanning**
- **Penetration Testing**
- **Log Retention Compliance**:
  - Verify 90+ day retention period
  - Test log access controls
  - Validate audit trail completeness

12. **UI/UX Testing**

- Date format validation
- Thousand separator and decimal format
- Timezone display verification
- Theme consistency (light/dark mode)
- Dropdown auto-focus functionality
- Form validation and alignment
- Grid column alignment
- Warning popup placement and design

Deliverables

- Test Plans (Sprint and Release level)
- Test Cases
- Test Scripts and Test Data
- Bug Reports with severity classification
- Test Execution Reports
- QA Metrics and Analytics
- Security Test Reports
- Performance Test Reports

Phase 6: DevOps & CI/CD

Owner: DevOps Team
Participants: Development Team, Security Team

Key Activities

1. **Version Control Setup**
    - Configure Git repository (GitHub/Bitbucket/AWS CodeCommit)
    - Define repository structure
    - Set up access controls
2. **Branching Strategy**
    - Select appropriate strategy (Git Flow/GitHub Flow/Trunk-Based)
    - Document branching rules
    - Configure branch protection
3. **Build Automation (CI)**
    - Set up continuous integration pipeline
    - Configure automated builds

- Integrate unit tests
- Add static code analysis (SonarQube)
- Include security scanning

4. **Deployment Pipelines (CD)**

    - Configure deployment automation
    - Set up approval gates
    - Implement deployment strategies (blue-green, canary)

5. **Separate Environments for Deployment & Testing**

    - **Development Environment**: For active development
    - **QA Environment**: Dedicated testing environment
    - **Staging/UAT Environment**: Pre-production validation
    - **Production Environment**: Live environment

6. **Environment Isolation**

    - Network segmentation between environments
    - Separate databases per environment
    - Environment-specific configurations
    - Access control per environment level
    - No production data in lower environments

7. **Infrastructure as Code (IaC)**

    - Use Terraform/CloudFormation/Ansible
    - Version control infrastructure code
    - Automate resource provisioning

8. **Semantic Versioning**
    - Implement Major.Minor.Patch versioning
    - Automate version increment
    - Tag releases appropriately
9. **Security Integration**
    - Automated vulnerability scanning
    - Dependency scanning
    - Secret management (no hardcoded credentials)
    - Security gates in pipeline
10. **Monitoring & Alerting**
    - Application performance monitoring
    - Infrastructure monitoring
    - Log aggregation
    - Alert configuration
11. **Release Rollback Strategy**
    - Document rollback procedures
    - Test rollback process
    - Maintain rollback capability for all components

Deliverables

- CI/CD Pipelines
- Isolated Environment Setup (4 separate environments)
- Deployment Strategy Documentation
- Infrastructure Code

- Security Scan Reports
- Monitoring Dashboards
- Rollback Procedures

Phase 7: UAT & Go-Live

Owner: Client Stakeholders
Participants: All Teams

Key Activities

1. **User Acceptance Testing**
    - Client stakeholders test in UAT environment
    - Validate business requirements
    - Test end-to-end scenarios
    - Provide feedback
2. **Production Checklist Review**
    - Verify all pre-deployment tasks
    - Security checklist completion
    - Performance benchmarks met
    - Documentation updated
3. **Pre-Go-Live Activities**
    - Create rollback plan
    - Prepare cut-off plan (if switching systems)
    - Create deployment plan with timeline
    - Establish communication plan
    - Conduct training sessions
    - Update documentation

4. **Deployment Execution**

    - Execute deployment plan
    - Monitor deployment process
    - Verify deployment success
    - Execute smoke tests

5. **Post-Deployment Verification**

    - Run production checklist tests
    - Verify critical functionality
    - Check integrations
    - Monitor system health

6. **Go-Live Support**

    - Provide immediate support
    - Monitor for issues
    - Address urgent problems
    - Communicate status to stakeholders

Deliverables

- UAT Sign-off
- Production Release Package
- Release Notes
- User Documentation
- Training Materials
- Go-Live Checklist

Phase 8: Post-Production Support

Owner: Support Team
Participants: Development Team, DevOps Team

Key Activities

1. **Hypercare Support**
    - Intensive monitoring period (typically 2-4 weeks)
    - Dedicated support team availability
    - Rapid issue resolution
    - Daily status reporting
2. **L2/L3 Technical Support**
    - **L2 Support**: Handle complex technical issues
        - Backend server access
        - Database troubleshooting
        - Advanced configuration
    - **L3 Support**: Development team involvement
        - Code-level issues
        - Architecture problems
        - Unknown/new problems
3. **Production Incident Management**
    - Incident logging and tracking
    - Severity classification
    - SLA adherence
    - Root cause analysis
    - Incident reporting

4. **Hotfix Management**

    - Rapid issue identification
    - Quick fix development
    - Emergency testing
    - Expedited deployment
    - Documentation update

5. **Maintenance & Support (KANBAN Model)**

    - Ongoing bug fixes
    - Minor enhancements
    - Performance optimization
    - Security patches
    - SLA compliance monitoring

6. **Continuous Monitoring**

    - Application performance
    - Error rates
    - User activity
    - Security events
    - Resource utilization

Deliverables

- Incident Reports
- Hotfixes
- Support SLA Compliance Reports
- Maintenance Logs
- Performance Reports

Security Controls & Compliance

Authentication & Access Control

Multi-Factor Authentication (MFA)

- Implement MFA for all user accounts
- Support multiple MFA methods (SMS, authenticator apps, hardware tokens)
- Enforce MFA for privileged accounts
- Allow MFA recovery options
- Track MFA adoption rates

Password Management

- **Password Policies**:
    - Minimum length: 12 characters
    - Complexity requirements (uppercase, lowercase, numbers, special characters)
    - Password expiration: 90 days
    - Password history: prevent reuse of last 5 passwords
    - Account lockout after failed attempts, depending upon client requirement.
- **Secure Storage**:
    - Hash passwords with strong algorithms (bcrypt, Argon2)
    - Salt passwords individually
    - Never store plaintext passwords

Session Management

- **Session Timeout**:
    - Idle timeout: 15-30 minutes (configurable)
    - Absolute timeout: 8-12 hours
    - Automatic session termination, depending on client requirement.
- **Session Security**:
    - Secure session token generation
    - HTTPOnly and Secure flags on cookies

  - Session token rotation after login
  - Invalidate sessions on logout
  - Concurrent session management

Role-Based Access Control (RBAC)

- Define clear user roles and permissions
- Implement principle of least privilege
- Regular access reviews
- Automated access provisioning/deprovisioning

Monitoring & Logging

Error Handling & Logging

- **Comprehensive Logging**:
  - Application errors and exceptions
  - Authentication attempts (success/failure)
  - Authorization failures
  - Session events
  - Data access logs
  - Configuration changes
  - Administrative actions
- **Log Retention**:
  - Minimum retention period 30 days
  - Long-term archival for compliance (1+ years)
  - Secure log storage with encryption
  - Restricted log access
- **Log Management**:

- Centralized log aggregation
- Log integrity protection
- Regular log review
- Automated alerting for critical events

Audit Trail

- Track all significant system events
- User activity monitoring
- Data modification history
- Configuration change tracking
- Compliance reporting

Development Security

Security Code Reviews

- **Review Checklist**:
  - Input validation and sanitization
  - Output encoding
  - Authentication and authorization
  - Session management
  - Cryptographic operations depending on client.
  - Error handling
  - Sensitive data handling
  - SQL injection prevention
  - Cross-site scripting (XSS) prevention
  - Cross-site request forgery (CSRF) protection
  - File upload security
  - API security

Static Code Analysis

- Automated scanning with different tools cypress
- Identify security vulnerabilities using different tools like code climate, new relic.

- Code quality metrics integrated with github repos.
- Technical debt tracking using tools like santry

Vulnerability Scanning

- Automated dependency scanning
- Container image scanning
- Infrastructure scanning
- Regular security assessments

Secure Coding Practices

- Follow OWASP Top 10 guidelines
- Security training for developers
- Use security libraries and frameworks
- Secure configuration management
- Secret management (no hardcoded credentials)

Environment Security

Separate Deployment Environments

- **Development**: Isolated development environment
- **QA**: Dedicated quality assurance environment
- **Staging/UAT**: Pre-production validation environment
- **Production**: Live production environment

Environment Isolation

- Network segmentation
- Separate databases per environment
- Environment-specific configurations
- No production data in lower environments
- Data masking for non-production environments

Access Controls

- Environment-specific access permissions
- MFA for production access
- Audit logging of environment access
- Regular access reviews
- Temporary access for contractors

Secure Change Management

- Formal change request process
- Change approval workflow
- Impact assessment
- Rollback procedures
- Change documentation
- Post-change validation

Quality & Performance Metrics

Code Quality Metrics

1. **Bugs per 1000 Lines of Code (LOC)**
    - Target: < 5 bugs per 1000 LOC
    - Track by severity
    - Trend analysis over time
2. **Code Coverage**
    - Target: 70-80% minimum
    - Unit test coverage
    - Integration test coverage

3. **Maintainability (SonarQube)**

    - Code smells
    - Technical debt ratio
    - Complexity metrics
    - Duplication percentage

4. **Security Vulnerabilities**

    - Critical vulnerabilities: 0 tolerance
    - High severity: remediate within 7 days
    - Medium severity: remediate within 30 days
    - Track OWASP Top 10 compliance

Testing Metrics

1. **Defect Density**

    - Formula: Defect Count / Size of Release (LOC)
    - Track per sprint and release
    - Categorize by severity

2. **Test Coverage Percentage**

    - Functional test coverage
    - Requirements coverage
    - User story coverage

3. **Pass/Fail Rate**

    - Test execution pass rate

- First-time pass rate
- Retest pass rate

4. **Defect Leakage**

- Formula: Defects in UAT / Defects in QA Testing
- Target: < 10%
- Root cause analysis

5. **Test Execution Time**

- Average time per test case
- Total test cycle time
- Automation time savings

Performance Metrics

1. **Response Times**

- Page load time: < 3 seconds
- API response time: < 500ms
- Database query time: < 100ms

2. **Resource Utilization**

- CPU usage: < 70% average
- Memory usage: < 80%
- Disk I/O
- Network bandwidth

3. **Scalability**

- Concurrent users supported
- Throughput (requests per second)
- Horizontal scaling capability

4. **MTBF/MTTR**

    - Mean Time Between Failures (MTBF)
    - Mean Time To Repair/Recovery (MTTR)
    - Availability percentage (target: 99.9%+)

Productivity Metrics

1. **Sprint Velocity**

    - Average story points per sprint
    - Velocity trend over time
    - Team capacity utilization

2. **Story Points**

    - Planned vs completed
    - Estimation accuracy
    - Story point burndown

3. **Release Frequency**

    - Number of releases per month
    - Time between releases
    - Release success rate

4. **Bug Lead Time**

- Time from bug creation to resolution
- By severity level
- Trend analysis

Security Metrics

1. **MFA Adoption Rate**

    - Percentage of users with MFA enabled
    - Target: 100% for privileged accounts
    - Overall target: 95%+

2. **Session Timeout Compliance**

    - Percentage of sessions properly terminated
    - Average session duration
    - Timeout configuration compliance

3. **Security Code Review Coverage**

    - Percentage of code with security review
    - Target: 100% for security-critical components
    - Security issues found per review

4. **Log Retention Compliance**

    - Verification of 30+ day retention
    - Log completeness percentage
    - Audit trail integrity

5. **Vulnerability Remediation Time**

- Time to fix critical vulnerabilities
- Time to fix by severity
- Vulnerability backlog

Key Workflows

User Story Workflow

States: New → Active → Resolved → Closed

1. **New**: Story created and added to backlog
2. **Active**: Story selected for sprint and development started
3. **Resolved**: Development complete, ready for testing
4. **Closed**: Testing complete, story accepted

INVEST Principles:

- **I**ndependent: No dependencies on other stories
- **N**egotiable: Open for discussion and refinement
- **V**aluable: Delivers value to user/customer
- **E**stimatable: Team can estimate effort
- **S**mall: Completable within a sprint
- **T**estable: Has clear acceptance criteria

Bug Workflow

States: New → Active → Resolved → Closed

Additional States: Duplicate, Won't Fix, Cannot Reproduce

1. **New**: Bug identified and logged
    - Include: Steps to reproduce, system info, severity
    - Assign priority and severity
2. **Active**: Bug assigned and work in progress

- Developer investigates and fixes
- Updates status regularly

3. **Resolved**: Fix implemented and ready for testing

- Deploy to test environment
- Notify QA team

4. **Closed**: Fix verified by QA

- Regression testing passed
- Documentation updated

Severity Levels:

- Critical: System down, data loss, security breach
- High: Major functionality broken
- Medium: Moderate impact on functionality
- Low: Minor issues, cosmetic problems

Release Process Workflow

Environments: Dev → QA → Stage/UAT → Production

1. **Development Environment**

- Feature development
- Unit testing
- Code reviews
- Integration testing

2. **QA Environment**

- Smoke testing
- Functional testing

- Regression testing
- Performance testing

3. **Stage/UAT Environment**

    - User acceptance testing
    - Final validation
    - Performance verification
    - Security testing

4. **Production Environment**

    - Deployment execution
    - Smoke testing
    - Monitoring
    - Go-live support

Release Checklist:

- [ ] All user stories completed and tested
- [ ] No critical or high severity bugs
- [ ] Performance benchmarks met
- [ ] Security testing passed
- [ ] Release notes prepared
- [ ] Rollback plan ready
- [ ] Stakeholder approval obtained
- [ ] Production checklist completed

Semantic Versioning

Format: MAJOR.MINOR.PATCH

Version Components:

- **MAJOR**: Significant features with breaking changes (e.g., 1.0.0 → 2.0.0)

- **MINOR**: New features without breaking changes (e.g., 1.0.0 → 1.1.0)
- **PATCH**: Bug fixes and hotfixes (e.g., 1.0.0 → 1.0.1)

Examples:

- First release: 1.0.0
- Add multi-tenancy: 2.0.0
- Add reporting feature: 2.1.0
- Fix critical bug: 2.1.1

Recommended Tools & Technologies

Project Management Tools

Tool	Description	Best For
Azure DevOps	Comprehensive PM and test management	Enterprise projects, Microsoft ecosystem
Jira	Extensive Agile process management	Agile teams, customizable workflows
ClickUp	Flexible project management	Small to medium teams
Monday.com	Visual project management	Cross-functional teams

Test Management Tools

Tool	Description	Best For
Azure DevOps Test Plans	Integrated test management	Teams using Azure DevOps
TestRail	Dedicated test case management	QA-focused teams
Zephyr	Jira-integrated testing	Teams using Jira

Automated Testing Tools

Tool	Purpose	Technology

Selenium	Web application testing	Java, C#, Python
Appium	Mobile application testing	iOS, Android
Cucumber	BDD testing	Gherkin syntax
Postman	API testing	REST APIs
JUnit/NUnit	Unit testing	Java, .NET
Cypress	End-to-end web testing	JavaScript

Code Quality & Security Tools

Tool	Purpose	Features
SonarQube	Static code analysis	Quality gates, security vulnerabilities
ESLint	JavaScript linting	Code style enforcement
Checkstyle	Java style checking	Coding standards compliance
PyLint	Python code analysis	PEP 8 compliance
Coverity	Security scanning	Vulnerability detection

CI/CD & DevOps Tools

Tool	Purpose	Features
Azure Pipelines	CI/CD automation	Build, test, deploy
GitHub Actions	Workflow automation	Event-driven pipelines
Jenkins	Build automation	Plugin ecosystem
GitLab CI/CD	Integrated CI/CD	Source control + pipelines

AWS CodePipeline	AWS-native CI/CD	AWS service integration

Infrastructure as Code (IaC)

Tool	Purpose	Best For
Terraform	Multi-cloud IaC	Cloud-agnostic infrastructure
AWS CloudFormation	AWS infrastructure	AWS-only deployments
Ansible	Configuration management	Server provisioning
Puppet	Infrastructure automation	Enterprise environments

Monitoring & Logging

Tool	Purpose	Features
Application Insights	APM for Azure	Performance monitoring
New Relic	Full-stack monitoring	End-to-end visibility
Datadog	Infrastructure monitoring	Metrics and logs
ELK Stack	Log aggregation	Elasticsearch, Logstash, Kibana
Splunk	Log analysis	Enterprise logging

Best Practices & Guidelines

Sprint Planning Best Practices

1. **Sprint Duration**: 2-3 weeks recommended
2. **Story Selection**: Based on priority and capacity
3. **Definition of Ready**: Stories must meet criteria before sprint
4. **Definition of Done**: Clear completion criteria for all stories

5. **Sprint Goal**: Single, focused objective per sprint

Code Review Best Practices

1. **Review Size**: Maximum 400 lines per review
2. **Review Time**: Within 24 hours of PR creation
3. **Checklist**: Use standardized review checklist
4. **Feedback**: Constructive and specific
5. **Security Focus**: Always include security considerations

Testing Best Practices

1. **Test Early**: Begin testing during development
2. **Automate**: Automate repetitive tests
3. **Test Data**: Use realistic, anonymized data
4. **Coverage**: Aim for 70-80% code coverage
5. **Documentation**: Keep test cases updated

Security Best Practices

1. **Least Privilege**: Grant minimum necessary access
2. **Defense in Depth**: Multiple security layers
3. **Regular Updates**: Keep dependencies current
4. **Security Training**: Regular team training
5. **Incident Response**: Have a plan ready

Deployment Best Practices

1. **Blue-Green Deployment**: Minimize downtime
2. **Canary Releases**: Gradual rollout to users
3. **Feature Flags**: Control feature activation
4. **Monitoring**: Watch metrics during deployment
5. **Rollback Plan**: Always ready to revert

Conclusion

This SDLC process framework provides a comprehensive approach to software development with strong emphasis on quality, security, and continuous improvement. By following these guidelines and adapting them to specific project needs, teams can achieve:

- Higher quality software products
- Improved security posture
- Better team productivity
- Enhanced stakeholder satisfaction
- Measurable continuous improvement

The framework is designed to be flexible and can be tailored to fit different project types, team sizes, and organizational requirements while maintaining core principles of quality and security.

Document Control

Version: 2.0
Last Updated: 2025
Document Owner: PMO / Development Leadership
Review Cycle: Quarterly
Next Review: [Set based on implementation date]

This document should be reviewed and updated regularly to reflect evolving best practices, new tools, and lessons learned from project implementations.